Loom — Manual

Loom is a browser-based, session-based music workstation. It organises sound into **lanes** (instrument tracks), **clips** (patterns of notes), and **scenes** (rows of clips launched together). Seven synthesis engines are available — TB-303, Subtractive, FM, Wavetable, Karplus-Strong, Sampler, and Drum Machine — and all synthesis runs live in the browser with no install or sign-in required.

Beyond the engines, Loom has **note velocity & dynamics** (an Ableton-style velocity lane), **per-clip and arrangement-wide loop regions**, **per-lane modulation and FX** with a sidechain mixer, **MIDI import**, ready-made **sampled drum kits**, a **playable arrangement** timeline, and optional **stem separation** (split a finished song into 4 lanes via a local helper).

The Loom interface: transport across the top, the session clip grid in the centre, and per-lane channel strips below.

Table of Contents

Loom-Manual.pdf

Live demo: https://ijol.github.io/Loom/

Getting Started

Open the app

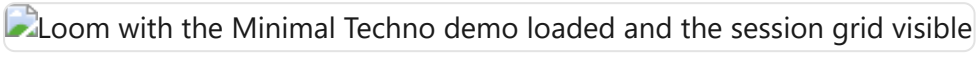
Loom runs entirely in the browser — no installation, no account, no plugins.

- **Live demo:** https://ijol.github.io/Loom/
- **Run locally:** clone the repository, then run `npm install` followed by `npm run dev`. The app is available at http://localhost:5173.

Make your first sound

When the app opens it loads the *Minimal Techno* demo automatically, so there is already a full arrangement ready to play.

1. Click the ▶ **Play** button in the transport bar. The first click resumes the browser's AudioContext — this is a browser requirement; audio cannot start without a user gesture, so nothing plays until you press Play.
2. Sound starts immediately. Use the **Volume** slider in the transport to adjust the output level.
3. Want to hear a different demo? Select one from the **— load a demo —** dropdown (Minimal Techno, Acid Rain, Cordillera, Neon Drive). The session loads and begins at the first scene.

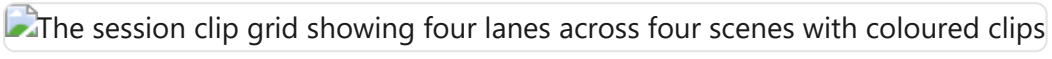


The full Loom interface. Transport across the top; coloured clips in the session grid; per-lane channel strips below.

The mental model

Loom organises music in three nested concepts:

- A **lane** is an instrument track. Each lane runs one engine (TB-303, Subtractive, FM, Wavetable, Karplus-Strong, Sampler, or Drum Machine) and has its own mixer strip with EQ, sends, pan, and fader.
- A **clip** is a pattern of notes that lives in a lane. A lane can hold multiple clips — one per scene row.
- A **scene** is a horizontal row across all lanes. Launching a scene fires the clip in each lane at that row simultaneously, so scenes work like song sections or loop variations.



The session grid: lanes run left to right as columns, scenes top to bottom as rows. Click a clip to select it; click the scene launch button on the left to fire the whole row.

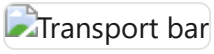
Click any clip cell to open its editor in the inspector below the grid. The editor shows a piano roll for melodic lanes and a drum grid for drum lanes.

Next steps

- Transport — BPM, meter, swing, and the unified REC controls (take / live-WAV / offline-WAV).
- Sessions, Lanes, Clips & Scenes — how to build and arrange your own session from scratch.

Transport

The header runs across the top of the interface in two rows. The first row (transport & tempo) holds the playback and timing controls you touch while the music plays; the second row (session & I/O) holds the mode toggle, recording/export, and session file management.



Play / Stop

Press ▶ (`#play` , title "Play — arranca el transporte") to start playback, and ■ (`#stop` , title "Stop — detiene el transporte") to stop it. Play and Stop are now two separate buttons — Play is no longer a play/stop toggle. When you press Play the browser's `AudioContext` resumes automatically if it was suspended (a browser policy requirement on first interaction).

Position and time readout

The readout shows two values side by side:

- **Bar.Beat.Step** (`#transport-position` , e.g. `1.1.1`) — the current song position derived from elapsed time, BPM, and the active meter. The counter resets to `1.1.1` on each new Play.
- **Elapsed** (`#transport-time` , e.g. `00:00:00`) — wall-clock time since the most recent Play press.

Both readouts update via `requestAnimationFrame` while playing and freeze on stop so you can read the final position. Because each lane has its own independent loop clock, the global position is computed from elapsed seconds rather than a single sequencer cursor. The visual playhead is similarly a display-only timer matched to scheduled audio time; it may drift slightly under browser tab throttling, but audio scheduling is unaffected.

Tempo controls

BPM (`#bpm`) — sets the tempo. Range: 40–240 BPM, default 130. You can type a value directly or use the number-input spinners. BPM changes propagate immediately to the sequencer, all lane engines, delay/LFO sync, and stretch-mode loop buffers. The change takes effect on the next scheduled step; it does not alter a note that is already held.

Each 16th-note step lasts `60 / bpm / 4` seconds.

Meter (`#meter`) — sets the global time signature. The dropdown offers common meters: 4/4, 3/4, 2/4, 5/4, 6/8, 7/8, 9/8, and 12/8. The meter controls how bars map onto the 16th-step grid and how the position readout counts beats. Changing the meter takes effect on the next loop cycle.

Swing (`#swing`) — intended to add a shuffle feel by delaying odd 16th-note steps. Range: 0 (straight) to 0.6. The value is saved and restored with the session. **Note:** swing is currently stored and persisted but is not yet read by the scheduler, so moving the slider does not change the timing you hear — full swing is planned for a future update.

Master volume

Volume (`#volume`) — the master output level, range 0–1 (default 0.5). This is a post-mix gain applied before the output visualiser.

Mode toggle and REC

Session / Performance (`#mode-toggle`) — switches the main view between the Session clip grid and the Performance arrangement view.

↟ **Copy to Performance** (`#copy-to-performance`) — an icon-only button (tooltip "Copiar las escenas a la timeline de Performance") that copies the current scenes onto the Performance timeline. See Performance & Arrangement.

- **Session** is the default mode: you see the clip grid and can trigger scenes, edit clips, and record automation in real time.
- **Performance** shows the arrangement timeline where recorded takes are displayed as timeline bands and automation curves can be drawn directly. See Performance & Arrangement for the full workflow.

● **REC** (`#rec` , tooltip "Grabar — el modo se elige al lado") arms recording; the **mode selector** (`#rec-mode`) beside it chooses what gets recorded:

- 🎛 **take** (`data-recmode="take"` , the default) — records knob moves and clip launches into a performance take. See Modulation & Note FX for how automation lanes work.
- ⏱ **live** (`data-recmode="live"`) — records the real-time audio output to a WAV.
- ⚡ **offline** (`data-recmode="offline"`) — renders the current scene to a WAV offline (fast).

Click **REC** again to disarm. This unified REC group replaced the old standalone WAV-export button.

Output visualiser

The canvas at the far right (`#viz`) shows a real-time waveform of the master output. It updates continuously while the page is open and gives a quick visual check that audio is flowing.

Session management, export, and demos

The remaining controls in the transport row are covered in dedicated chapters:

- **WAV export** — there is no longer a standalone "↓ WAV" button. WAV export now happens through the **REC** group's mode selector (see the Mode toggle and REC section above): ⏱ **live** records real-time audio to WAV, and ⚡ **offline** renders the scene to WAV offline (fast). See Saving & Export.
- **New / Save / Load** (`#new-session` , `#save` , `#load`) — session file management. See Saving & Export.
- **— load a demo —** (`#demo-picker`) — loads a bundled demo arrangement into the session.
- ▶ **MIDI IMPORT** (`.midi-panel`) — imports a Standard MIDI File. See the MIDI Import chapter for details.
- ☰ **Stems…** (`#stems-open`) — opens the stem-separation dialog, which splits a finished song into four Sampler lanes (vocals / drums / bass / other) via a local helper service. Requires the service to be running; see MIDI & Samples → Stem separation.

Sessions, Lanes, Clips & Scenes

The session view is the main workspace in Loom. It organises everything you hear into a compact grid of lanes and clips that you can launch, edit, and rearrange in any order.


Session view — toolbar and grid

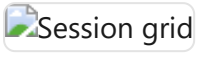
The model

Lane — one instrument track. Each lane runs its own synthesis engine (TB-303, Drums, Subtractive, FM, Wavetable, Karplus, or Sampler) with its own mixer strip and insert chain. Lanes appear as columns in the grid.

Clip — a pattern of notes that lives at one lane × scene cell. Every clip stores a list of note events, an optional name, a colour, and a length in bars. A cell is either filled (it holds a clip) or empty.

Scene — a row in the grid. Launching a scene fires one clip per lane simultaneously, aligning all starts to the same quantize boundary so the whole arrangement snaps together cleanly.

The grid


Session grid

Lane names run along the top header row; row numbers (1, 2, 3 …) label the scenes down the left edge. Scenes launch buttons sit in the rightmost column. Each filled cell shows the clip's name (or its row number as a fallback) against the clip's pastel colour. A ⚙ icon in the lane header opens that lane's instrument editor.

A newly created instrument lane starts out empty — no placeholder clips are added to its column; you fill cells yourself. Audio and Sampler lanes created from a dropped WAV or loop place their clip in row 1 only. The grid always keeps at least one launchable scene available even when every lane is empty.

Right-clicking grid elements opens a context menu. On a **lane header**: *Editar instrumento*, *Parar pista*, and *Borrar pista* (red). On a **scene cell**: *Lanzar escena*, *Añadir escena*, and *Borrar escena* (red). On a **filled clip**: *Abrir editor*, *Reproducir / Parar*, and *Borrar clip* (red). On an **empty cell**: *Crear clip* — on audio lanes this entry is disabled and instead reads *Importar audio (arrastra un WAV)*.

Below the scene rows there is a stop row: each lane has its own ■ stop button, and a global ■ **all** button at the far right stops every lane at once.

Deleting from the grid

A small ✕ cross appears on every lane header (*Borrar pista*), every filled clip cell (*Borrar clip*), and every scene cell (*Borrar escena*). Deleting a clip is immediate. Deleting a lane or scene asks you to confirm in an in-app dialog (**Aceptar** / **Cancelar**, with the destructive choice in red) only when the target still has content — an empty lane or scene is removed straight away. Every deletion is undoable with **Ctrl+Z**, which restores the lane, scene, or clip along with its audio resources.

Launching clips and scenes

Click the ▶ **icon** on any filled cell to launch that clip. The icon switches to ⏸ once the clip is playing; clicking ⏸ stops that lane. Clicking anywhere else on the cell body — outside the play icon — opens the inspector without affecting playback.

When the transport is stopped, launching a clip starts it immediately. When the transport is running, the launch is held until the next quantize boundary, then the clip starts in sync with the rest of the session.

Click the ▶ **N** button at the right of a scene row to launch all clips in that row together. All lanes share the same boundary, so they start aligned. The toolbar's ■ **All** button stops every lane at once.

The quantize boundary is determined by the clip's own `Quantize` setting (see the inspector below). If the clip has no override, the lane's quantize applies; if the lane has none, the session's global quantize is used.

The inspector

Clicking a clip cell body opens the inspector below the grid with controls for that clip.

Control	What it does
Name	Free-text label shown on the cell.
Length (bars)	How many bars the clip plays before looping.
Quantize	Per-clip launch alignment: *Default* (inherits lane/global setting), *Immediate* (no wait), or a fixed interval — 1/4, 1/2, 1 bar, 2 bars, or 4 bars.
Copy notes	Copies the clip's notes to an internal clipboard.
Paste ▸ Replace	Replaces the selected clip's notes with the clipboard contents.
Paste ▸ Layer	Merges the clipboard notes into the selected clip rather than replacing them.
Duplicate	Creates a copy of the clip in the next empty cell of the same lane.
↔ Editor	Toggles between the piano-roll editor and the drum-grid editor. The appropriate editor is chosen automatically based on the lane's engine; use this button to switch if needed. See Editing Clips for the full editing reference.
🎲 **Notes**	Randomizes the clip's note content using the current scale and root settings.
Delete	Removes the clip from the grid (also triggered by Delete/Backspace while the inspector is open).

The editor embedded inside the inspector is rebuilt each time a new clip is selected; deep editing of notes is covered in Editing Clips.

Arranging clips

You can move a clip to any other cell by dragging it. Hold **Ctrl** while dragging to copy instead of move. Clips remember their colour across moves and copies. Drag a clip onto a cell that already has a clip to swap or replace it.

Each clip is assigned a colour automatically from a pastel palette. You can distinguish patterns at a glance even before you give them names.

For the mixer strip, insert effects, and per-lane send levels, see Mixing & FX. For the available synthesis engines on each lane, see Engines.

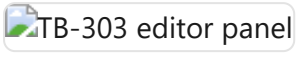
Engines

Every lane in Loom runs exactly one synthesis engine. You choose the engine with the ENGINE selector at the top of the lane's editor panel. Changing the engine replaces the sound source while preserving the lane's clips and modulation routing.

Each engine exposes a PRESET dropdown (Load / Save As / Delete) and a 🎲 **Sound** button that randomises the patch and sets the preset name to "Custom". Presets are JSON assets stored per-engine; they include GM programme tags so that MIDI import can auto-assign the best engine and preset for each track. See Sessions, Lanes, Clips & Scenes for how to add and configure lanes.

Every engine responds to **note velocity**. Velocity (0–127) scales each note's loudness continuously; accent (velocity ≥ 100) layers additional character on top — brightening the filter envelope and resonance on bass-style engines, and adding brightness on drums. For how to view and edit velocities in the piano-roll or drum-grid, see Velocity & dynamics.

TB-303



Above: TB-303 editor — Wave, Cutoff, Resonance, Env Mod, Decay, Accent, and a per-lane LFO.

The TB-303 is a monophonic, resonant bass synthesiser modelled on the Roland TB-303. It is the natural choice for acid bass lines but works equally well for aggressive leads and any sound that calls for a steep, self-oscillating filter sweep.

TB-303 parameters

Parameter	Description
Wave	Sawtooth or Square oscillator waveform
Cutoff	Filter cutoff frequency (0–100%)
Resonance	Filter Q — self-oscillates at high values
Env Mod	How far the filter envelope opens the filter per step
Decay	Filter envelope decay time
Accent	Per-step level: brightens the filter, bumps Q, raises output gain

Slide and accent behaviour

A note's `slide` flag means "slide into the next step". When the scheduler emits step N it checks whether step N-1 carried a slide flag; if so, it ramps the pitch from the previous note and skips the amp re-attack so the gate stays open across the boundary. The outgoing step gets an extended gate (1.5× step length) so the overlap is audible.

Accent is a per-step flag that simultaneously brightens the filter envelope, raises the resonance Q, and boosts the output gain — the classic 303 bassline punctuation technique.

The engine ships with 20+ presets from "BASS Acid Classic" to "LEAD Squelch". See Editing Clips for how to set slide and accent on individual steps.

Subtractive

Above: Subtractive editor — OSC 1/2, Sub oscillator, Noise, Filter (with built-in envelope), Amp, and POLY controls.

The Subtractive engine is a classic analogue-style polyphonic synthesiser with two oscillators, a sub oscillator, a noise source, a resonant low-pass filter, and a full amplitude envelope. It is the most general-purpose engine in Loom and suits pads, leads, basses, and plucks.

Parameter sections

- **OSC 1 / OSC 2** — waveform (Saw/Sqr/Tri/Sin), level, and detune in cents. Detuning the two oscillators creates the classic "supersaw" chorus effect.
- **SUB / NOISE** — sub oscillator level (one octave below OSC 1) and a noise generator for breath and texture.
- **FILTER** — Cutoff, Resonance, Env Amount, Drive, Key Track, and a full ADSR filter envelope (toggle with Built-in Env).
- **AMP** — Attack/Decay/Sustain/Release amplitude envelope (toggle with Built-in Env).
- **MASTER** — global Tune in semitones.
- **POLY** — voice count (1–16), poly/mono mode, and legato/retrig behaviour in mono mode.

For modulation routing see Modulation & Note FX.

FM

Above: FM editor — Algorithm selector, Op 1–4 (Ratio/Detune/Level/ADSR), global Mix and Voices.

The FM engine is a four-operator, DX7-style frequency-modulation synthesiser. Each operator is a sine oscillator with its own ADSR amplitude envelope and level; operators are wired together according to one of four algorithms.

FM parameters

Parameter	Description
Algorithm	1 = serial 4→3→2→1; 2 = three parallel mods → Op 1; 3 = two pairs; 4 = additive
FB (Op4)	Op 4 self-feedback — adds odd harmonics and edge
Op 1–4: Ratio	Frequency ratio relative to the played note (0.1–16×)
Op 1–4: Det	Per-operator detune in cents
Op 1–4: Level	Carrier output level or modulation index (modulators)
Op 1–4: ADSR	Per-operator amplitude envelope
Mix	Final output level (0–1)
Voices	Polyphony cap (1–16; default 6)

FM suits metallic tones, electric pianos, bells, and evolving textures. Small ratio changes yield very different timbres; the preset library covers bells, organs, and electronic basses.

Wavetable

Above: Wavetable editor — Wave A/B selectors, Morph, Detune, Filter, Amp envelope, and Voices.

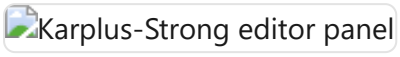
The Wavetable engine morphs between two pre-computed waveforms — Wave A and Wave B — using the Morph knob or an LFO/ADSR routed to it. Both waves are drawn from a fixed bank of eight anti-aliased tables: Sine, Triangle, Sawtooth, Square, Pulse (25%), Organ, Brass, and Vocal.

Wavetable parameters

Parameter	Description
Wave A / Wave B	Source waveforms to interpolate between
Morph	Crossfade position (0 = full Wave A, 1 = full Wave B)
Detune	Global detune in cents
Cutoff / Res	Resonant low-pass filter
Built-in Env	Toggle the built-in amp ADSR on/off
Attack / Decay / Sustain / Release	Amplitude envelope
Voices	Polyphony cap (1–16; default 8)

Animating Morph with an LFO is the signature technique — sweeping from Sine to Sawtooth or Brass to Vocal while a note sustains produces evolving, living tones. See Modulation & Note FX.

Karplus-Strong



Above: Karplus-Strong editor — String section (Damping, Brightness), Excite section (Excite time, Noise Tone), and Amp controls.

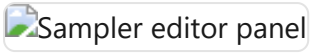
Karplus-Strong is a physical-modelling engine that synthesises plucked-string sounds. Loom renders each note offline (sample-by-sample in JavaScript) into a buffer and plays it back through an amplitude envelope. This gives exact pitch at every frequency, natural high-harmonic roll-off, and no feedback runaway.

Karplus-Strong parameters

Parameter	Description
Damping	T60 decay: 0 = long sustain (~4 s), 1 = muted (~0.12 s)
Brightness	Loop filter: 0 = dark/cello, 1 = open/metallic
Excite	Excitation burst length (pluck sharpness)
Noise Tone	Colour of the excitation noise (dark → bright)
Attack / Release	Amp envelope on buffer playback
Level	Output amplitude
Voices	Polyphony cap (1–16; default 8)

Damping and Brightness are set per-note at the moment of the pluck (baked into the buffer). Level and its envelope are live AudioParams and can be modulated. Karplus suits acoustic bass, guitar, harp, and marimba-style sounds.

Sampler



Above: Sampler editor — global Gain/Voices controls; per-pad controls appear once samples are loaded.

The Sampler engine plays back audio samples mapped across the keyboard. Each keymap zone (pad) has its own per-pad parameters read at trigger time, making it possible to tune, filter, and pan individual pads independently.

Global parameters

Parameter	Description
Gain	Master output gain for the lane
Voices	Polyphony cap (1–16; default 8)

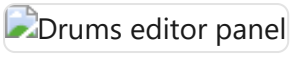
Per-pad parameters

Shown in the drum-voice rack once pads are mapped.

Parameter	Description
Tune	Transposition in semitones (−24 to +24)
Cutoff / Res	Per-pad resonant low-pass filter
Attack / Decay	Per-pad amplitude envelope
Level	Per-pad output level
Pan	Stereo position
Rev / Dly	Send amounts to the shared reverb and delay
Loop / Loop Start	Loop mode (one-shot or loop-while-gated) and loop start point
Retrig	Poly (voices overlap) or Mono (re-hit cuts the previous voice)

A Sampler lane with pads mapped to GM drum note numbers automatically enters drumkit mode and shows the drum-grid editor instead of the piano roll. For details on loading samples and building keymaps see MIDI & Samples.

Drums (Drum Machine)



Above: Drums editor — Preset row, master bus knobs (Vol/Pan/Rev/Dly/Lo/Mid/Hi), per-voice rack with Tune/Decay/Rev/Dly controls per voice.

The Drums engine is a fully synthesised eight-voice drum machine — no samples required. All eight voices (Kick, Snare, Closed Hat, Open Hat, Clap, Cowbell, Tom, Ride) are built from oscillators, noise generators, and simple envelopes. Each voice routes through its own channel strip and then to a shared drum bus.

Voice synthesis rack

The per-voice rack exposes the key parameters for each voice:

Voice	Key parameters
Kick	Tune, Attack (click), Decay, Start/End Freq, Sweep, Wave
Snare	Tune, Tone body, Snap (noise), Body Decay, Noise Decay, Noise Tone
Closed Hat	Tune, Decay
Open Hat	Tune, Decay
Clap	Tone, Decay, Sharp (filter Q)
Tom	Tune, Decay, Sweep, Start/End Freq
Cowbell	Tune, Decay, Detune
Ride	Tune, Decay

Drum preset dropdown — synth kits and sample kits

The preset dropdown for any drum lane lists kits from three groups:

Group	Kits	How it works
GM	KIT Standard, KIT Room, KIT Power, KIT Electronic, KIT TR-808, KIT Jazz, KIT Brush, KIT Orchestra	GM-programme aliases that map to the synth kits below
Synth	TR-909, TR-808, TR-606, CR-78, LinnDrum	100% synthesised DSP — no samples required
Samples	TR-808 (samples), Acoustic / Dirt (samples), Dirt (samples)	Real one-shot WAVs bundled with Loom

Synth kits seed all eight per-voice parameters from the kit's characteristic values; you can edit individual voices on top and hit 🎲 Sound to randomise all voices at once.

Sample kits load the matching WAV for each voice (kick, snare, closed hat, open hat, clap, tom, cowbell, ride) from `public/drumkits/` and rebuild the keymap fresh on every session load — you never need to re-import the files manually. Once a sample kit is selected, the lane uses the drum-grid editor and the full per-pad parameter rack, exactly like a Sampler lane in drumkit mode.

The sample WAVs are curated one-shots from the Dirt-Samples collection (used by TidalCycles) and classic TR-808 recordings. Full credits are in the repo `README.md` under "Credits — sample sources".

> Note: sample kits are loaded by the Sampler engine under the hood. For the Sampler's own Instrument family selector (Melodic / Percussion / Loop) and per-pad parameters, see MIDI & Samples.

Bus controls

The master bus row gives Vol, Pan, Rev, Dly, Lo, Mid, and Hi (±18 dB shelves) for the whole drum bus. These are automatable via Modulation & Note FX. Routing to the shared reverb and delay sends is covered in Mixing & FX.

Summary table

Engine	Best for	Standout parameters
TB-303	Acid bass lines, resonant leads	Slide, Accent, Env Mod
Subtractive	Pads, leads, basses, general-purpose	Dual OSC detune, Filter Drive, Key Track, POLY mode
FM	Bells, electric pianos, metallic textures	Algorithm, per-operator Ratio, FB feedback
Wavetable	Evolving tones, digital leads, pads	Morph (A→B crossfade), 8-waveform bank
Karplus-Strong	Plucked strings, guitar, harp	Damping (sustain), Brightness, Excitation
Sampler	Any audio, drum kits with per-pad control	Per-pad Tune/Filter/Envelope, Loop mode
Drums	Synthesised drum machine, percussion	8-voice synth rack, kits-as-presets, bus EQ

Editing Clips

Every clip in Loom holds a sequence of notes. To edit those notes, click the **body** of any filled cell in the session grid — anywhere except the ▶ play icon or the × delete cross in its corner (clicking × deletes the clip outright, with no confirmation). You can also **right-click** a filled cell and choose **Open editor**. The inspector panel opens below the grid and the editor renders inside it. Closing the inspector does not stop playback — launching and editing are independent.

Melodic lanes (TB-303, Subtractive, FM, Wavetable, Karplus, Sampler) open the **piano-roll**. Drum-machine lanes and sampler lanes that have a drum kit loaded open the **drum-grid**. If you want to switch between the two views for a given clip, click the ↔ **Editor** button in the inspector toolbar.

See Sessions, Lanes, Clips & Scenes for how clips are organised, and Engines for the controls each engine exposes.

Piano-roll

The piano-roll is a two-axis canvas: time runs left-to-right, pitch runs bottom-to-top. A vertical keyboard on the left names the rows; a time ruler at the top marks bars and beats.

Zoom and pan

Scrub **vertically on the time ruler** to zoom the time axis; scrub **horizontally** on the ruler to pan. Scrub the **keyboard strip** vertically to zoom the pitch axis. The native scroll bars pan both axes. Zoom state is saved per clip and restored when you reopen it.

Draw mode (pencil)

The default tool is **Draw**. Click an empty area of the grid to place a note at the snap resolution (default: 16th notes). Drag right while placing to extend its duration. Click and drag an existing note's right edge to resize it. Click and drag the body of a note to move it. Alt-click or right-click a note to delete it.

Select mode

Click the **Select** tool button in the toolbar to switch. In Select mode, click a note to select it. **Drag on the grid background** to draw a marquee rectangle; every note whose body intersects the rectangle is selected. Click empty space to deselect all.

With notes selected you can:

- **Move as a group** — drag any selected note; the whole selection moves together and is clamped to the clip boundaries.
- **Delete** — press Delete or Backspace.
- **Nudge** — use the arrow keys to shift the selection one snap unit left/right or one semitone up/down.
- **Cut / Copy / Paste** — Ctrl+X / Ctrl+C / Ctrl+V (Cmd on Mac). Paste anchors the earliest clipboard note at the mouse cursor position, so move the pointer where you want the paste to land before pressing Ctrl+V.

The tool choice and clipboard contents persist across clip re-opens and across clips, so you can copy from one clip and paste into another.

Computer-keyboard note input

When the piano-roll has focus you can record notes directly from the computer keyboard:

Key row	Notes
`a s d f g h j k` (home row) — white keys	C D E F G A B C
`w e t y u` (upper row) — black keys	C# D# F# G# A#
`z` / `x`	Shift input octave down / up

Pressing a key **auditions the note** immediately so you can hear it, advances the input cursor by one snap step, and **records the note into the clip** at the cursor position. Duration equals one snap step (quantised on key-up). This lets you step-enter a melody quickly without a MIDI controller.

Drum-grid

The drum-grid is a canvas editor where rows correspond to drum voices (kick, snare, hi-hat, etc.) and columns correspond to time positions. Each hit is placed at a precise tick within the clip.

Grid resolution

A resolution selector at the top of the editor sets the snap and the column width. Available resolutions are:

Value	Description
1/4	Quarter notes
1/8	Eighth notes
1/8T	Eighth-note triplets
1/16	Sixteenth notes (default)
1/16T	Sixteenth-note triplets
1/32	Thirty-second notes
free	No snap — place hits at any tick

You can mix resolutions across clips; each clip stores its own `gridResolution` setting.

Placing and removing hits

Click an empty cell to place a hit. Click an existing hit to remove it. In **free** mode, click anywhere in the row — the hit lands at the exact tick under the pointer with no snapping.

Selection and group operations

Drag on the canvas background to draw a marquee rectangle. Hits whose row and time position intersect the rectangle are selected. With a selection active:

- **Move** — drag horizontally to shift selected hits in time; the group is clamped to the clip length.
- **Move rows** — drag vertically to reassign hits to different drum voices; the relative row offsets are preserved and clamped to the available voice list.
- **Delete** — press Delete or Backspace.
- **Cut / Copy / Paste** — Ctrl+X / Ctrl+C / Ctrl+V. Paste anchors the earliest hit at the click position (tick × row), preserving relative offsets within the group.

Selection, clipboard, and group-move all operate on **row indices**, not MIDI numbers, so patterns copy cleanly even between kits that map voices to different MIDI notes.

Playhead

A vertical playhead line moves across the canvas in real time while the clip is playing, driven by the sequencer's look-ahead clock. It updates on every redraw tick and resets to the left edge when the clip stops.

Loop regions

A **loop brace** sits above every clip editor — piano-roll and drum-grid — as a narrow strip spanning the full clip length. It lets you mark an A–B sub-region and repeat just that portion while the clip plays.

Setting the loop region

The brace strip has two drag handles (left = A, right = B) and a **Loop** toggle button. To use it:

1. Click **Loop** to enable the loop region. The region highlights between the A and B handles; if no region was set before, it defaults to the full clip length.
2. Drag the **left handle** to move the start point (A). Drag the **right handle** to move the end point (B). Both handles snap to 16th-note grid positions.
3. While the clip is playing, the scheduler repeats only the A–B sub-region — the rest of the clip is skipped.

Click **Loop** again to disable it. The clip reverts to playing its full length; the A and B positions are remembered so you can re-enable the same region later.

What the loop brace affects

The loop region works the same way for all clip types:

- **Note clips (piano-roll)** — only notes whose start falls within the A–B range are triggered; the period of repetition equals the duration of that range.
- **Drum clips (drum-grid)** — same: only hits inside the sub-region fire.
- **Sliced note clips (Sampler)** — a clip whose notes trigger bank slices behaves like any note clip: only the hits inside the A–B sub-region fire, still tempo-locked and pitch-preserving.

The brace lives on the note/drum editor. A pure audio channel (the waveform-only audio-clip editor) does not show a brace; to gain per-region control over an audio loop, bring it in through the Sampler's **Loop** family, which slices it into a note clip with a piano-roll (see MIDI & Samples — Sampler).

The loop region is **per-clip** and saved with the session. Two clips in the same lane or scene can each have their own independent A–B region, or none at all.

All loop-region edits (moving handles, toggling) are part of the global undo history (Ctrl+Z / Cmd+Z).

For how to use an arrangement-wide A–B loop brace that repeats a section across all lanes at once, see Performance & Arrangement.

Waveform header

Any clip that references an audio buffer shows a **waveform header** above its editor — a peak view of the buffer with a bar/beat ruler, slice markers (orange), and a live playhead. It appears in two situations:

- **Audio clips** open the dedicated **audio-clip editor** (toolbar + waveform header, no note grid).
- **Sliced note clips** (and any normal clip that carries a display-only waveform reference) keep the header **above** the piano-roll or drum-grid, so you can edit notes against the source audio.

To bring a loop into a session as a tempo-locked audio channel, see MIDI & Samples — Audio channel (the **+ Audio** control, the waveform header, the **Warp** tempo-lock). To chop a loop into individually editable note slices, load it through the Sampler's **Loop** family instead.

Velocity & dynamics

Above: piano-roll editor. The strip beneath the note grid is the velocity lane — one vertical bar per note, height proportional to velocity.

Every note in Loom carries a **velocity** value from 0 to 127. Velocity is set when you draw a note (default: **90**), adjusted in the velocity lane, and captured automatically from MIDI import. It affects the sound in two complementary ways:

- **Loudness** — velocity scales the note's output gain continuously via a smooth curve (`velToGain`). A velocity of 1 is near-silent; 127 is the loudest a note can be. Notes at the default of 90 sit just above the mid-point of the range, so there is clear headroom in both directions.
- **Accent character** — notes with velocity **≥ 100** are accented. On top of the continuous gain, accent adds character to the sound: on bass-style engines (TB-303, Subtractive) it brightens the filter envelope and raises the resonance Q; on drums it increases brightness. This is the same accent model that the 303 bassline and drum sequencer have always used, now unified into the velocity scale.

Reading velocity visually

Each note's **fill colour** shifts along a **blue** → **yellow** ramp as its velocity increases. Low-velocity notes are deep blue; high-velocity notes are warm yellow. The transition is weighted so the blue half of the range covers roughly velocities 0–64 and the yellow half covers 64–127. Accented notes (≥ 100) are additionally outlined with a **white border** — colour alone does not distinguish accent from non-accent.

The velocity lane

Below the note grid (piano-roll) or the drum-voice rows (drum-grid) is the **velocity lane**: a row of vertical bars, one per note, anchored at the note's start position. Bar height is proportional to velocity. A **dashed horizontal line** across the lane marks the accent threshold (velocity 100) so you can see at a glance which notes are accented. The lane scrolls horizontally in sync with the grid.

Editing velocities

You interact with the velocity lane by dragging the bars:

- **Set a single note** — drag a bar up or down. The velocity updates live; the note colour and audible gain change as soon as you release.

- **Adjust a group** — if you have notes selected (marquee selection in the main grid), dragging any bar that belongs to the selection applies the **same delta** to all selected notes. Notes that would go out of range are clamped to 1–127.
- **Paint a ramp** — drag horizontally across multiple bars. Each bar you pass over is set to the velocity corresponding to the current vertical position of the pointer, writing a smooth velocity ramp across the passage in a single gesture.

When several notes share the same start position (a chord), their bars are fanned a few pixels apart in the lane so each one remains individually grabbable.

All velocity edits are undoable (Ctrl+Z / Cmd+Z) in the same undo history as note placement and movement.

Modulation & Note FX

Every lane has two signal-processing layers that sit *before* its engine's audio output and *around* its note stream: **Modulators** shape sounds over time by continuously driving engine parameters, and **Note FX** transform the note events before they reach the engine. Both are per-lane and saved with the session.


FM engine editor showing the MODULATORS and NOTE FX sections *The FM engine editor. The MODULATORS section (LFO1 + ADSR1) and the NOTE FX row are visible at the bottom of the panel.*

Modulators

Open any lane's engine editor and scroll to the **MODULATORS** section. Press **+ LFO** or **+ ADSR** to add a modulator; you can add as many as you need. Each modulator appears as a card with its own controls, an **ON / OFF** toggle, and a **×** remove button.

LFO

An LFO generates a periodic waveform that you route to one or more target parameters.

Control	Description
WAVE	Waveform shape: Sine, Tri, Sqr, or Saw
RATE	Free rate in Hz (0.01–40 Hz, default 4 Hz)
RATIO	Musical division for BPM-sync mode (e.g. 1/4, 1/8, 1/8T, 1/16.)
FREE / SYNC	Toggles between free-running Hz rate and BPM-locked ratio. In FREE mode the RATE knob is shown; in SYNC mode the RATIO selector replaces it.
POLARITY	**-1..+1** (bipolar, default) oscillates symmetrically around the param's centre. **0..1** (unipolar) only pushes the parameter upward.
TRIG	**Free** — the LFO phase runs continuously regardless of note triggers, like a classic analogue LFO. **Note** — phase resets on every note-on, so the LFO peak always lands at the start of each note. Hidden when scope is PER-VOICE (see below).
SCOPE	**Shared** — one LFO voice runs for the whole lane. **PerVoice** — an independent LFO is spawned for each played note and lives for the duration of that note.

Musical difference between Shared and PerVoice: a shared LFO on a polyphonic lane makes all voices wobble together in phase — the whole chord breathes as one. PerVoice gives each note its own independent modulation cycle, so fast staccato chords shimmer individually rather than in unison.

ADSR

An ADSR produces a classic Attack–Decay–Sustain–Release envelope that fires once on each note trigger and follows the note's gate duration. Its default scope is **PerVoice**, which is almost always what you want: every note gets its own independent envelope.

Control	Range	Default
A (Attack)	1 ms – 2 s	10 ms
D (Decay)	1 ms – 4 s	300 ms
S (Sustain)	0 – 100 %	70 %
R (Release)	1 ms – 8 s	300 ms

You can switch an ADSR to **Shared** scope if you want a single envelope that re-triggers on each note but whose shape is not tied to any individual voice. In practice PerVoice is the natural choice for most envelopes.

Destinations and depth

Below each modulator card's controls is a destination list. To route the modulator:

1. Select a target from the dropdown — it lists all automatable parameters for the lane's engine, plus any lane inserts, master inserts, and master sends (reverb, delay).
2. Press **+ Destination**. A new row appears showing the target name and a **DEPTH** knob (range –1 to +1, default 0.5).

The depth knob scales the modulator's effect. A value of 1.0 means the full output range of the modulator sweeps the full range of the destination parameter (in the parameter's native units). –1.0 inverts the modulation direction. Multiple destinations can share the same modulator at different depth values.

Remove a destination with its **×** button. Remove the whole modulator with the **×** on the card header.

Note FX

The **NOTE FX** section sits directly below MODULATORS. Note FX processors intercept the lane's note stream before it reaches the engine, transforming which notes are played and when. Add a processor with **+ Arp** or **+ Chord**. Each processor shows an **ON / OFF** button and a **×** remove button.

Note FX are per-lane and persist with the lane's engine state. Loading a demo resets them to the demo's configuration.

Arpeggiator

The Arp takes each held note and generates a rapid sequence of notes from it according to a scale and direction pattern.

Control	Options / Range	Description
PATTERN	up, down, updown, random, cosmic	Direction of the arpeggio. **cosmic** adds occasional octave jumps and random steps for an unpredictable feel.
SCALE	major, minor, pentMinor, phrygian, chromatic	Scale from which arp notes are drawn, starting from the held root note.
RATE	free, 1/4, 1/8, 1/8t, 1/16, 1/16t, 1/32	Step rate in musical divisions (BPM-synced), or free Hz.
OCT	1–4	Number of octaves the arp spans. Higher values cycle the scale across more octaves before repeating.
GATE	0.05–1.0	Fraction of the step interval during which each arp note is held. Lower values create a more staccato feel.
FREE Hz	0.5–32 Hz	Rate used when RATE is set to *free*.

The Arp fires as many notes as fit inside the original note's gate duration, so longer notes produce longer runs.

Chord

The Chord processor replaces each incoming note with a chord built on that note as the root.

Control	Options	Description
CHORD	maj, min, maj7, min7, sus2, sus4, dim	Chord voicing to generate
OCT	–2 to +2	Octave offset applied to all notes in the chord

All notes in the chord share the original note's timing and gate length.

Automation

Loom records parameter automation in two ways:

Real-time knob recording (Performance view): in the session/I-O header row, make sure the REC mode selector beside ● **REC** is set to 🎛 **take** (the default), then press ● **REC** to arm recording and press Play. While recording in take mode, every knob you move *and* every clip launch is captured as automation in the current take. Automation is written at a sub-step resolution derived from the BPM. Press ● **REC** again to disarm. (The other two REC modes — ⏱ **live** and ⚡ **offline** — export WAV audio instead of recording automation; see Transport for the unified REC group.)

Per-clip envelopes (Session view): each clip can carry automation lanes independent of the Performance take system. Open the inspector for a clip and scroll below the note editor to the automation section. Select a parameter from the dropdown and click the add button to create an

envelope lane for that clip. The envelope draws a curve over the clip's length (in bars) and plays back each time the clip loops. Envelopes are stored alongside the clip's notes in the session file.

When you move a clip to a lane running a different engine, envelopes whose parameter ID no longer exists in the new engine are disabled automatically (they remain in the clip but do not play back until re-enabled or deleted).

See also: Engines for the parameters you can modulate, Mixing & FX for lane inserts and master FX whose parameters also appear as modulation destinations, and Transport for the REC button and Performance view.

Mixing & FX

Every lane in Loom has its own signal path from the synthesis engine through to the master output. This chapter explains how that path is structured, what controls are available per lane, and how the shared Master FX panel ties everything together.

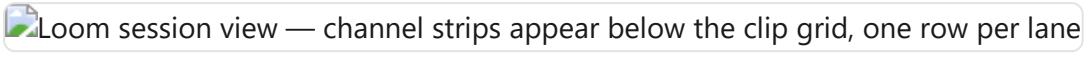
Signal flow overview

```
Lane engine → lane insert chain → channel strip (EQ → comp → level → pan → mute → duck) → maste
                                                                                  └──→ reverb send
                                                                                  └──→ delay send
master bus → master insert chain → master compressor → output                     FxBus returns —
```

In short: the engine's audio passes through any per-lane inserts first, then the channel strip where EQ, sends, pan, and level are applied. The processed signal joins the master bus, which runs through the master insert chain and the master compressor before reaching the speaker.

Per-lane channel strip

Each lane owns a `ChannelStrip`. Its controls are visible below the session grid in the lane's row.


Loom session view — channel strips appear below the clip grid, one row per lane

Level (fader)

The vertical slider sets the lane's output gain as a linear multiplier (0–1 = silence to unity; the percentage label reflects the current value). This is the last gain stage before output, applied after EQ and the per-lane compressor.

Pan

The **PAN** knob positions the lane in the stereo field. Centre (0) is the default; turning it left or right continuously shifts the image. The pan value is automatable and can be modulated — see Modulation & Note FX.

Mute and Solo

M silences the lane by zeroing its mute gain node. **S** solos the lane: all other lanes are muted in the UI while solo is active. Both controls affect what the sidechain tap feeds downstream — a muted lane's tap still carries pre-mute signal so sidechain routing remains stable.

3-band EQ

The three EQ knobs apply before the per-lane compressor:

Knob	Filter type	Centre frequency	Notes
LO	Low-shelf	200 Hz	Boost or cut lows; default ±0 dB
MID	Peaking	1 000 Hz	Q = 1; adds presence or scoops the midrange
HI	High-shelf	4 500 Hz	Boost or cut highs and air

All three bands are ±dB adjustments. EQ gain AudioParams are exposed to modulation so you can automate filter sweeps from the modulation panel.

REV and DLY sends

The **REV** and **DLY** knobs control how much of this lane's post-duck signal is fed into the shared master reverb and delay returns. They are independent wet levels: 0 = dry only, higher values mix the lane into the reverb or delay tail. The shared reverb and delay processors live in the Master FX panel; per-lane send amounts just determine how much goes in.

Per-lane inserts

Every lane also has a private insert chain that sits *before* the channel strip — the engine's audio passes through it first. If you open the lane's inspector and add FX to its insert list, those effects process the lane signal exclusively and do not affect any other lane.

Inserts vs sends: an insert is a serial in-line processor (distortion, filter, etc.) that replaces the signal passing through it; a send is a parallel path that adds wet signal from a shared return (reverb, delay). Use inserts when you want a destructive or tone-shaping effect on a single lane; use sends when you want a common acoustic space that multiple lanes share.

The insert types available per lane are the same four plugins used on the master chain — see Master FX panel below for their parameter details.

Master FX panel

The master bus has its own strip at the foot of the **scenes column** of the mixer row — a **MASTER** label, an **FX** button, a fader that mirrors the master **Volume**, and a VU meter. Click its **FX** button to open the **Master FX panel** below the grid (click again to close it). *(Previously this was a separate "Master FX" tab; the controls are identical, only the way you open them changed.)*


Loom Master FX panel — SENDS, MASTER COMP, and INSERTS sections

SENDS — Reverb and Delay

The SENDS section holds the global return effects. Lane REV/DLY knobs feed into these processors; the knobs here control the shared effect itself.

REVERB parameters:

Param	Range	Description
Wet	0–1.5	Wet output level
PreD	0–0.5 s	Pre-delay before the reverb tail starts
Size	0.05–8 s	Impulse response length (room size)
Decay	0.1–10	Tail decay shape (higher = longer tail)

The reverb is a convolution reverb with a procedurally generated impulse response. Size and Decay rebuild the impulse in real time when adjusted.

DELAY parameters:

Param	Range	Description
Time	0.01–2 s	Delay time (BPM-synced at session start: 3/8 beat × BPM)
Fbk	0–0.95	Feedback amount
Wet	0–1.5	Wet output level
Damp	200–12 000 Hz	Low-pass filter on the feedback loop; lower values darken repeats

MASTER COMP

The master compressor sits at the tail of the master chain, after all inserts. It uses the same `CompBlock` as the per-lane strip compressor, so the parameters are identical:

Param	Range	Default	Description
Bypass	on/off	on	Pass-through when on
Threshold	−100 to 0 dB	−24 dB	Level above which compression starts
Ratio	1–20	4	Compression ratio
Attack	0–1 s	0.003 s	Gain reduction onset time
Release	0–1 s	0.25 s	Gain recovery time
Knee	0–40 dB	30 dB	Transition softness around the threshold
Makeup	~0–4 (linear)	1	Post-compression gain, up to about +12 dB

The master compressor is bypassed by default. Enable it for glue and loudness control on the final mix, or to tame transient peaks before export. See Saving & Export for how the master bus feeds the offline render.

INSERTS → Master Filters

Below MASTER COMP, the INSERTS section holds the master insert chain. Click **+ Add Filter** to append a slot. Each slot can hold one of four plugin types, selectable in the slot's Type control:

Filter (multifilter)

- Type: LP / HP / BP / Notch
- Freq: 20–20 000 Hz (exponential)
- Q: 0.1–24

Distortion (Dist)

- Drive: 0–1 — waveshaper saturation amount (4x oversampled)
- Mix: 0–1 — dry/wet blend

Reverb — same parameters as the send reverb above (Wet, PreD, Size, Decay). Use as an insert to apply reverb to the full master rather than via sends.

Delay — same parameters as the send delay (Time, Fbk, Wet, Damp). Use as an insert for a master-bus slapback or stutter.

Slots in the chain are ordered in series: the output of each slot feeds the input of the next. Each slot has a bypass toggle so you can A/B it without removing it. Individual slots can be removed; adding the same type multiple times is allowed.

The master insert chain is distinct from the FxBus send effects — insert slots process the full mixed signal, while the send reverb and delay receive per-lane amounts and return to the master bus independently.

Sidechain compression

Loom includes a sidechain ducking system. Any lane's channel strip can be ducked by the signal level of another lane (the *source*). The ducker follows the source lane's envelope via a full-wave rectifier and two smoothing filters, then reduces the target lane's gain proportionally:

```
duckGain ≈ 1 − depth × env(source)
```

Sidechain parameters (set per lane in the lane inspector):

Param	Range	Default	Description
Source	lane selector	—	Which lane's post-mute tap drives the duck
Depth	0–1	0.6	How deep the gain dips at full envelope
Attack	s	0.005 s	How quickly the ducker opens (gain rises)
Release	s	0.25 s	How quickly the ducker closes (gain falls)
Threshold	dB	−40 dB	Source envelope must exceed this to duck at all

A typical use case is kick-drum ducking: set a bass or pad lane's sidechain source to the kick lane. Every kick hit momentarily ducks the bass, creating a pumping effect common in electronic music. Because the tap is taken from the source lane post-mute (but pre-duck), muting the source stops the ducking without feedback loops.

The sidechain bus is separate from the compressor block available on each channel strip. The per-lane compressor (`CompBlock`) is a standard dynamics compressor in the signal path; the sidechain ducker is a parallel envelope-follower that modulates gain. Both can be active simultaneously.

For engine-level sound design that feeds the channel strips, see Engines. For LFO and ADSR modulation of EQ, sends, pan, and other AudioParams, see Modulation & Note FX.

MIDI & Samples

This chapter covers two ways to bring external material into a Loom session: importing a Standard MIDI File to populate lanes and clips automatically, and loading audio samples into the Sampler engine to build melodic instruments or drum kits.

MIDI Import

The MIDI Import panel lives in the toolbar at the top of the screen, next to the demo picker. Click **MIDI Import** to expand it.

Loading a file

Click the file picker and choose a `.mid` or `.midi` file. Loom reads the file immediately — no server, no upload. The parser extracts every track's name, General MIDI programme number, and all note-on/note-off pairs (converted to start tick, duration, MIDI note, velocity, and channel). Only the first tempo event in the file is used; if no tempo is present, the session BPM stays as it is.

Empty tracks (those with no note events) are silently skipped.

The track list

After parsing, a row appears for each non-empty track showing:

- A **checkbox** to include or exclude that track from the import.
- The **track index**, track name (cleaned of control characters), note count, pitch range, and programme number.
- A **preset dropdown**. The GM programme number is looked up against every engine's preset catalogue, and the matching presets are offered at the top of the list (e.g. programme 33 "Acoustic Bass" will surface TB-303 and Subtractive bass presets). Below a divider, every preset from every engine is available so you can override the suggestion freely.
- A **▶ audition button** that plays a short three-note arpeggio through the currently selected preset without touching the session. Use this to compare options before committing.

Importing

Click **Import MIDI**. A confirmation dialog asks whether to **Add** or **Replace**:

- **OK (Add)** appends the imported lanes and a new scene to the current session. The new clips are placed on the new scene row so they line up correctly with the scene's launch button.
- **Cancel (Replace)** clears the session and seeds it with only the imported content.

Loom creates one lane per selected track. The lane's name is taken from the matched preset (e.g. "TB Bass"), while the clip inside it keeps the original track name. If the file contained a tempo, the session BPM is updated to match. The import launches the new scene immediately.

The conversion scales MIDI ticks to Loom's internal grid (based on quarter notes divided into four 16th steps), so the notes land on the correct beats regardless of the file's tick resolution.

See Engines for what each engine sounds like, and Sessions, Lanes, Clips & Scenes for how to rearrange lanes and scenes after import.

Sampler

The Sampler is a polyphonic playback engine that maps audio files across the keyboard and plays them back at the correct pitch per note. It has three **families**, chosen from an **Instrument** selector: **Melodic** (one or more zones spanning a range of keys), **Percussion** (single-note pads at GM drum notes — a drum kit), and **Loop** (a sliced loop played as notes). The inspector shows **GAIN** and **VOICES** knobs, a **Keymap** section, the **Instrument** family selector, and **Import samples…** / **Import loop…** controls.

Loading samples

Click **Import samples…** and pick one or more audio files (the picker is multi-select). Loom decodes each file and stores it in IndexedDB, so the samples persist across browser reloads — you do not need to re-import them each session.

Each imported file becomes a keymap entry that spans the full keyboard (MIDI 0–127) with the root note set to middle C (MIDI 60) by default. When you import several at once they stack full-range — last match wins, so only the last sounds until you narrow the ranges. Set each zone's root and low/high boundary in the keymap list, and remove an entry with the ✕ button.

Keymap and repitch

A **keymap entry** has a root note, a low-note boundary, and a high-note boundary. When a note falls within the range, the sample plays at a playback rate of `2^((midi - rootNote) / 12)`, giving equal-temperament repitching. Multiple entries can cover different key ranges — last match wins — so you can build multi-sample instruments by importing several files and adjusting their boundaries.

Per-pad / per-zone parameters

Every keymap entry (pad in drumkit mode, zone in melodic mode) has its own set of parameters, all read at trigger time:

Parameter	Range	Default	Description
TUNE	−24 to +24 st	0	Pitch offset in semitones, applied on top of keymap repitch
CUTOFF	0–1	1	Lowpass filter cutoff (0 ≈ 60 Hz, 1 = fully open)
RES	0–1	0	Filter resonance
ATTACK	0.001–2 s	0.005 s	Amplitude envelope attack time
DECAY	0.005–4 s	0.08 s	Release tail after the gate closes
LEVEL	0–1.5	1	Pad output level
PAN	−1 to +1	0	Stereo pan position
REV	0–1	0	Send level to the lane's reverb insert
DLY	0–1	0	Send level to the lane's delay insert
LOOP	Off / On	Off	When On, the sample loops while the gate is held
LSTART	0–1	0	Loop start point as a fraction of sample duration
RETRIG	Poly / Mono	Poly	Mono cuts the previous hit on re-trigger; Poly layers them

In drumkit mode these appear in the per-pad rack (the same eight-column layout used by the Drum Machine engine). In melodic mode they appear as a knob row below each keymap entry.

Percussion family (drum kits)

Pick the **Percussion** family in the **Instrument** selector and choose a kit ("— none (own keymap) —" leaves the lane on its own keymap). Loom fetches the kit's manifest, downloads each voice's WAV, stores them in IndexedDB, and maps every sample to its canonical General MIDI drum note (kick on 36, snare on 38, and so on). The keymap is rebuilt fresh from the manifest on each session load, so you do not need to re-import the kit files manually.

In addition to any kits you build yourself by loading samples, Loom ships three **ready-made sample drum kits** that appear directly in the Percussion family: **TR-808 (samples)**, **Acoustic / Dirt (samples)**, and **Dirt (samples)**. These are curated one-shot WAVs bundled with the app, so they work on the live GitHub Pages deploy without any manual file import. Simply pick one from the dropdown and the lane is ready to play.

Once a kit is loaded the lane switches to the drum-grid editor (the same grid used by the Drum Machine engine). You get per-pad mute and solo buttons, and the full per-pad parameter rack. To return to a melodic keymap, pick the **Melodic** family.

See Editing Clips for how to draw patterns in the drum grid, and Engines for a comparison with the Drum Machine engine (which lists all available kits, including the synth kits, in a unified preset table).

Audio channel

The **audio channel** is the first-class way to bring a finished loop into a Loom session: drop a WAV and it plays **tempo-locked to the project without changing pitch**, with its waveform shown as a header above the clip editor. It stays a pure audio loop; to chop a loop into individually editable note slices, load it through the Sampler's **Loop** family instead (see Sampler).

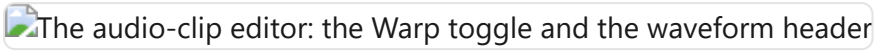
Creating an audio channel

There are two ways to add one:

- **+ Audio button** — at the end of the lane tab row, next to the engine picker, sits a **+ Audio** button. Click it and pick a WAV. Loom decodes the file, stores it in IndexedDB (so it survives a reload), estimates its original tempo, and creates a **new audio lane** holding the loop as an audio clip. The clip opens automatically in the inspector.
- **Drag onto an audio-lane cell** — once an audio lane exists, you can drag another WAV directly onto one of its grid cells to place a second audio clip there.

Each new audio lane gets a launch button on its scene row, so it is immediately playable alongside the rest of the session.

The audio-clip editor



An audio clip has no note grid. Clicking it opens the **audio-clip editor** — a ♺ **Warp ON / OFF** toggle (tempo-locking, see below) above a **waveform header**.

The waveform header shows a peak view of the buffer with a bar/beat ruler, any detected slice markers (orange), and a live playhead while the clip plays. This same header also appears **above** the normal piano-roll or drum-grid for any clip that references a buffer, so you always see the audio you are editing against.

Tempo-lock (Warp)

With **Warp ON** (the default), the audio channel plays in time with the project BPM using a pitch-preserving WSOLA time-stretch:

- **At the loop's native BPM** the stretch ratio is ≈ 1, so playback is essentially identical to the source file.
- **At any other project BPM** the buffer is time-stretched to fit — faster or slower — **without changing pitch**. The stretched buffer is cached and re-rendered automatically whenever you change the project tempo, so the loop stays locked as you experiment.

With **Warp OFF** the loop plays at its natural speed with no tempo sync — useful when you want the audio exactly as recorded.

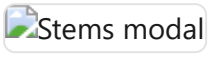
> First-play note: on the very first loop iteration after import (before the stretch cache is warm) playback briefly falls back to a varispeed render — a slight pitch shift that self-heals from the next iteration. At the loop's native tempo the ratio is ≈ 1, so even that first pass is near-identical.

Slicing a loop into notes

The audio channel itself is a pure WAV loop. To chop a loop into individually editable hits, load it through the Sampler's **Loop** family (see Sampler) rather than the audio channel. Loom detects slice points (from embedded **Acid /** `cue` **/ AIFF** markers when present, or by onset detection plus a tempo estimate), stores one short sample per slice in IndexedDB, and creates a **note clip** that triggers the slices in order on a piano-roll — so the groove plays back identically, now as discrete, editable notes. Move, mute, repitch, or re-order the hits in the piano-roll, and tweak each pad's tune/cutoff/decay/level/pan in the per-pad rack (see Per-pad / per-zone parameters); the clip keeps the original waveform as its header. *(Earlier builds did this from a **✂ Slice → pads** button on the audio channel; that moved to the Sampler's Loop family.)*

Stem separation (optional, local service)

Stem separation lets you drop a finished song into Loom and get it back as four separate Sampler lanes — **Vocals**, **Drums**, **Bass**, and **Other** — so you can mute, solo, and remix each part inside the existing session.



How it works

Click **☰ Stems…** in the session bar (the second header row, alongside Save / Load / MIDI). A dialog titled "Separate into stems" opens and immediately checks whether the local helper service is reachable:

- **Service found** — the hint line reads "4 tracks (Vocals / Drums / Bass / Other) via the local service." and the **Separate** button becomes active once you pick a file.
- **Service not found** — the hint reads "Can't find the stems service at localhost:8765. Is it running?" and Separate stays disabled. Start the service (see below), then re-open the dialog.

To separate a track: pick an audio file with the file picker, then click **Separate**. The dialog shows a progress bar:

1. **"Uploading…"** — the file is being uploaded to the local service.

2. **"Separating… m:ss"** — the service is running Demucs; the counter shows elapsed time. The bar may be indeterminate if the model does not report fine-grained progress.
3. On success the dialog closes automatically and four new Sampler lanes appear in the session — one per stem, each holding a full-length one-shot clip sized to the song. Hitting Play reconstructs the original mix; mute or solo any lane to isolate parts.

The entire lane-creation is a **single undo step**, so you can undo all four lanes at once.

Cancelar aborts a running job and frees the temporary files on the service. **Cerrar** closes the dialog (only available when no job is running).

Opt-in nature

The feature is entirely opt-in. If you never start the service, nothing else in Loom changes — the ☰ Stems… button is the only touch point, and it degrades gracefully to a clear "service not found" message.

Stems land in IndexedDB as ordinary sample assets: they survive browser reloads just like any other sample you import.

Setting up the local service

The separation runs on your machine via a small Python service in `tools/stem-service/`. It requires **Python 3.10+** and **ffmpeg** on your PATH.

```
cd tools/stem-service
python -m venv .venv
# macOS / Linux:
. .venv/bin/activate
# Windows:
.venv\Scripts\activate

pip install -r requirements.txt
uvicorn app:app --port 8765
```

The first time you separate a track the service downloads the Demucs `htdemucs` model automatically (several hundred MB). Subsequent runs skip the download. Separation takes roughly **1–2 minutes per song** on CPU; a GPU-enabled PyTorch build is much faster.

Codespaces and custom service URL

If you want to run the service in a GitHub Codespace (a Linux VM with Python), start it there with the same commands above, forward port 8765, and paste the resulting HTTPS URL into the browser console:

```
localStorage.loomStemServiceUrl = 'https://your-codespace-url-8765.preview.app.github.dev'
```

The same override works for any non-default host. CORS for `localhost:5173` (dev), `localhost:4173` (preview), and the GitHub Pages origin is already configured in the service.

> Note: Chrome's *Private Network Access* policy may add a preflight request when the Pages version of Loom calls `http://localhost:8765`. The lowest-friction setup is running Loom locally (`npm run dev`) alongside the service.

For full notes on CORS, the HTTP contract, and the Codespaces workflow, see `tools/stem-service/README.md`.

Saving & Export

Loom keeps everything in the browser. There is no account, no cloud, and no server upload — your sessions live in your browser's `localStorage` and on your own filesystem when you export them. This chapter covers how to save and restore sessions, undo your work, and render a scene to a WAV file.

Sessions in the browser

Every parameter, every lane, every clip, and every scene is part of the current session. Changes take effect immediately; nothing is auto-committed to disk. When you close the tab, the browser preserves the last-saved state via an autosave entry in `localStorage`, so reopening Loom typically drops you back where you left off.

Three buttons on the session bar (the second header row) drive session management:

Button	What it does
🗋 **New**	Discards the current session and starts a blank, empty one (tooltip "Nueva sesión vacía").
Save	Opens the Save Manager with the name field ready to type.
Load	Opens the Save Manager to browse and restore a saved session.

See Transport for the full two-row header layout (transport & tempo on top, session & I/O below).

Save Manager

Clicking **Save** or **Load** opens the Save Manager modal. It is the single place where all session persistence happens.

Saving a session

Type a name in the text field at the top and click **Save current** (or press Enter). Loom writes the full session state — BPM, time signature, every lane's engine + inserts + clips + scenes, the mixer state, and the arrangement take if one exists — into `localStorage` under a unique key, and also updates the autosave slot. The entry appears in the list immediately.

The save format is versioned (`schemaVersion: 3`). When you load an older file Loom migrates it automatically; saves with an unrecognised schema version are rejected with a warning rather than loading broken state.

The saved-session list

Each row in the list shows the session name, date/time, and size in KB. Per entry you can:

- **Load** — restores the session and closes the modal.
- ⤓ (download) — exports the entry as a `.json` file to your filesystem without closing the modal.
- ✎ (rename) — prompts for a new name in place.
- 🗑 (delete) — confirms and removes the entry.

The topmost row is **Auto-save (latest)**, which always reflects the state at the time of the last named save.

Load from file…

Imports a `.json` file you previously downloaded (or received from someone else). Loom validates the schema before applying it; an invalid file shows an alert.

Clear all saves

Removes every named entry from `localStorage`. The autosave slot is preserved. A confirmation dialog appears before anything is deleted.

Storage readout

The footer shows the total size of all named saves, so you can keep an eye on `localStorage` usage.

Undo / redo

Loom maintains a global undo stack that covers all session mutations: adding or removing lanes and clips, editing notes in the piano roll or drum grid, changing engine parameters via knobs, and more. The keyboard shortcuts are:

Action	Shortcut
Undo	Ctrl+Z / Cmd+Z
Redo	Ctrl+Shift+Z / Cmd+Shift+Z or Ctrl+Y

Knob gestures are bracketed — Loom snapshots the state at the moment you grab a knob and commits when you release, so a single drag is a single undo step. The shortcuts are inactive when a text input has focus, so typing a save name does not accidentally undo your work. Loading a session clears the undo history.

WAV export

WAV export is part of the unified **REC** group on the session bar (second header row), not a separate button. The ● **REC** button (tooltip "Grabar — el modo se elige al lado") records using whichever mode is selected in the adjacent **mode selector** (`#rec-mode`):

- 🎛 **take** (default) — records knob moves and clip launches into a performance take (not a WAV).
- ⏱ **live** — records the live master output in real time to a stereo 16-bit WAV file.
- ⚡ **offline** — renders the current scene to a WAV file offline (faster than real time).

To export audio, pick ⏱ **live** or ⚡ **offline** and press ● **REC**. The two WAV backends behave as described below.

⏱ live (real-time WAV)

The real-time backend is the ground-truth render. Loom restarts the scene from the top, taps the live master output (after all inserts, compression, and master FX), captures the audio through an `AudioWorklet` recorder, and stops the transport when the capture finishes. What you hear during the export is exactly what ends up in the file — the same signal path, the same timing, the same random variation from any voice that uses it.

Because it captures live audio, it takes exactly as long as the scene itself.

⚡ offline (fast WAV render)

The offline backend rebuilds the full audio graph — lanes, inserts, master bus — inside an `OfflineAudioContext` , applies every lane's current sound state, batch-schedules all note events, and renders faster than real time without touching the live session. It shares the same encoder and download step as the real-time path, so the output format is identical.

Offline rendering is faster but it replicates the deterministic part of the signal path; any non-deterministic element (e.g. a voice with random modulation at note-on) may differ from the live sound.

What gets exported

- **One pass of the scene.** The export captures one full iteration of the longest clip across all sounding lanes. Shorter clips in the same scene loop to fill that window, matching runtime behaviour. There is no infinite-loop export.
- **FX tail.** A 2-second tail is appended after the music ends so reverb decay and delay repeats are not cut off abruptly.
- **A scene must be playing.** If no scene is active when you press ● **REC** in a WAV mode (⏱ live or ⚡ offline), Loom shows a brief notice and does nothing. Launch a scene first (see Sessions), then export.
- **File name.** The download is named `loom-scene-<timestamp>.wav` .

After the export finishes the transport stops.

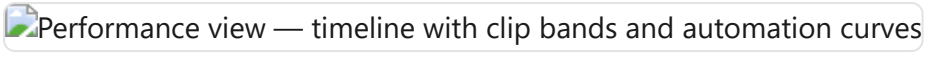
Live build and GitHub Pages

The public instance of Loom is deployed automatically to https://ijol.github.io/Loom/ — every push to `main` triggers a GitHub Actions workflow that runs `vite build --base=/Loom/` and deploys the result to GitHub Pages. The standard `npm run build` (base `/`) is for local development or self-hosting on any other path.

Performance & Arrangement

Loom has two main views: **Session** and **Performance**. The Session view is the clip grid you work in most of the time — lanes, clips, and scenes launched on demand. The Performance view is a linear **arrangement timeline**: a fixed song laid out from left to right with a playhead that moves forward through it, starts at the beginning, and stops at the end.

Switch between the two views using the **Session / Performance** toggle in the transport bar (`#mode-toggle`). Switching stops playback. See Transport for the full transport layout.


Performance view — timeline with clip bands and automation curves

---

Three ways to fill the arrangement

The arrangement starts empty. There are three ways to give it content.

1. Copy to Performance

The fastest route from a working session to a playable song is the ↧ **Copy-to-Performance** button (`#copy-to-performance`) in the session bar of the header — now an icon-only button with the tooltip "Copiar las escenas a la timeline de Performance". Clicking it calls `arrangementFromSession`, which walks your scenes in order and lays them out as a linear song:

- Each scene becomes one section.
- The section length equals the longest effective clip in that scene (measured in bars). If a clip has a loop sub-region enabled, its sub-region length is used instead of the full clip length.
- Every lane that has a clip assigned for that scene gets a timeline band covering the section.

After the layout is computed, Loom switches you to Performance automatically. The operation is undoable.

2. Record a take live

You can record the arrangement in real time while you play.

Recording is driven by the unified **REC** control in the session bar, which has three modes selectable beside the ● **REC** button: 🎛 **take** (the default — captures knob moves + clip launches into a performance take, described below), ⏱ **live** (records real-time audio to a WAV file), and ⚡ **offline** (renders the current scene to WAV offline, faster than real time). The steps below assume 🎛 **take** is selected.

1. Make sure the REC mode selector (`#rec-mode` , next to the REC button) is set to 🎛 **take** — its default. (The other two modes, ⏱ **live** and ⚡ **offline**, record audio to WAV instead; see Saving & Export.) Then click ● **REC** (`#rec`) in the session bar to arm recording.
2. Stay in Session view and press Play. Recording begins.
3. Launch clips and scenes as you would for a performance. Move any knobs whose automation you want captured.
4. Press Stop. Loom finalises the take: any still-open clip events are clamped to the stop time, durations are computed, and the recorded content appears in the Performance view as timeline bands and automation curves.

If you arm REC and then switch to Performance mode before pressing Play, the arm is cleared automatically (a toast notification appears) because Performance mode drives playback from the arrangement directly rather than from the live session.

3. MIDI import

When you import a Standard MIDI File via the **MIDI Import** panel (see MIDI & Samples), Loom calls the same `arrangementFromSession` logic after building the session. Because an imported MIDI file produces a single scene whose clips span the full song, the arrangement comes out as one long section per lane — the complete track laid out linearly from bar 1.

The timeline

Once the arrangement has content, the Performance view shows:

- **Toolbar** — Length (bars), Zoom slider, automation brush (Line / Flat), Loop A–B toggle, and a readout showing total bars and BPM.
- **Ruler** — a bar-numbered ruler across the top. When the A–B loop is active, a loop brace with two drag handles sits on the ruler.
- **Clip bands** — one row per lane. Each recorded or copied clip event is shown as a coloured block, labelled with the clip name, spanning the bars it occupies.
- **Automation curves** — below each clip band, any recorded or drawn automation curves appear. Each curve corresponds to one parameter (identified by its ID). You can draw into curves directly using the Line or Flat brush.
- **Master automation** — any automation curves routed to global (master) parameters appear in a separate section at the bottom.
- **Playhead** — a vertical line (`#perf-playhead`) that moves in real time via `requestAnimationFrame` while the arrangement is playing.

The timeline does not support moving or resizing bands by hand. Bands are generated by recording or Copy to Performance; to change the layout, re-record or re-copy.

A–B loop brace

The **Loop A–B** button in the Performance toolbar toggles the arrangement-wide loop brace. When active:

- Two drag handles on the ruler mark the loop start (A) and loop end (B). Drag either handle to set the window; handles snap to whole bars.
- The playhead wraps within [A, B). When it reaches B, all lanes are stopped and playback re-anchors to A instantly. Any clip that was already active spanning A is relaunched at the wrap point so there is no gap.
- When Loop A–B is off, Loom plays in **song mode**: the arrangement runs from the beginning to `durationSec` and stops — every lane is halted and the transport stops.

This loop brace operates on the arrangement timeline as a whole. It is distinct from the **per-clip loop brace** available inside the piano-roll and drum-grid editors, which loops a sub-region within a single clip. See Editing Clips for the per-clip loop.

Song playback

Press Play while in Performance mode to start playback from the beginning of the arrangement. The arrangement's own play state is used — it is independent of the live Session runtime. The playhead advances, clips launch at their scheduled times, and automation curves are applied continuously.

At the end of the arrangement (when Loop A–B is off), all lanes stop and `onArrangementEnd` fires, which stops the transport. You can press Play again to restart from the top.

Knob automation written into the arrangement's curves is applied every lookahead tick alongside clip launches. Automation values are normalised (0–1) internally and mapped to each parameter's min–max range at playback time.

Automation lanes

Automation curves can be added by hand — you do not have to record a take first. Set a non-zero **Length** in the toolbar (or record/copy content so the arrangement has a duration), then use the automation header that appears just below the ruler.

Adding a lane

The header row contains a grouped parameter dropdown and a **+ Automation** button. The dropdown lists every automatable parameter in the project, organised by prefix (lane ID or `master`). Each entry shows the parameter ID and its label — for example `lane-1.fx.reverb.wet — WET`. Select the parameter

you want, then click **+ Automation**. A new lane appears below the clip band for that lane (or in the Master section for global parameters). The curve starts flat at the parameter's current value.

Drawing the shape

Two brush buttons in the toolbar control how you paint:

- **Line** — click and drag to draw a ramp between the start and end points of the gesture. Use this for smooth fades, filter sweeps, or any gradual change.
- **Flat** — paints a constant value across the drag range. Use this for step-style automation or to hold a value steady across a section.

The active brush is highlighted. Each lane also exposes **On / Off** and **Smooth / Stepped** toggles in its header. **On / Off** mutes the curve without deleting it. **Stepped** switches interpolation from smooth linear to staircase, snapping the value at each sub-step boundary — useful for parameter jumps that should be instantaneous.

Removing a lane

Click the **×** button on the right side of the lane header to remove the curve entirely. The action is undoable with Ctrl+Z / Cmd+Z.

How automation curves play back

Curves added manually behave identically to curves captured by recording. During arrangement playback they are applied continuously alongside clip launches — knob values are updated every lookahead tick, mapped from the normalised 0–1 curve to the parameter's min–max range. Curves generated by a recorded take (see Transport — REC) and manually drawn curves live in the same list and can coexist on the same lane.

For modulator-driven per-lane automation (LFO / ADSR) that runs in Session view rather than the arrangement timeline, see Modulation & Note FX.

Persistence

The arrangement is saved as part of the session file (schema version 3). Recorded takes — clip bands, automation curves, loop brace position — survive Save / Load and browser restarts. Undo/redo covers arrangement edits (length, zoom, adding/removing curves, drawing automation, Copy to Performance) but not the raw recording; a take is finalised on Stop and does not participate in the undo stack.

See Saving & Export for how sessions are saved and loaded.

Developer Guide

This chapter is for contributors who want to extend Loom or understand how its internals fit together. Read it alongside `CLAUDE.md` at the repo root, which is the canonical, always-current architecture reference — this chapter expands on it in prose.

The spine

Three structures hold everything together:

1. **A plugin registry** — engines, FX, and modulators are all plugins. They are discovered at build time via a Vite `import.meta.glob` scan, so adding a new one means dropping a file in the right directory, not editing core wiring.
2. **`SessionState`** — the pure data model: lanes contain clips, scenes reference which clip each lane plays. No audio side-effects live here.
3. **`LaneResourceMap`** — owns the live Web Audio nodes for each lane. One entry per lane, holding a `ChannelStrip`, a `SynthEngine` instance, and an `InsertChain`. The lane allocator in `src/app/lane-allocator.ts` is the sole path for creating and swapping these resources; nothing else should construct them directly.

The plugin registry

`src/app/plugin-bootstrap.ts` calls `import.meta.glob` at build time over two trees:

```
src/engines/*.ts          — synth engines
src/plugins/fx/*.ts       — FX inserts
src/plugins/modulators/   — LFO / ADSR modulators
```

Every module in those trees is eagerly imported. Any exported value that satisfies the `PluginFactory` shape (`{ kind, manifest, create }`) is collected and registered via `registerPlugin`. The engine registry (`src/engines/registry.ts`) supports both a **singleton** pattern (`registerEngine`) for shared instances and a **factory** pattern (`registerEngineFactory` / `createEngineInstance`) for per-lane instances that need independent state.

`listEngines()` reads from the singleton map and is the source of metadata (name, type, polyphony, parameter specs) used to populate the lane engine selector. Engines declare their parameters as `EngineParamSpec[]`; voices expose continuous `AudioParam`s via `getAudioParams()`, and engines expose shared ones via `getSharedAudioParams()`. That is the entire surface the modulation and automation systems need — nothing else.

SessionState data model

`src/session/session.ts` defines three levels:

- **`SessionLane`** — has an `engineId`, a list of `SessionClip | null` slots, and an `engineState` bag that persists knob values, modulator configs, note-FX, sampler keymap, pad params, and kit mode.
- **`SessionClip`** — holds `notes: NoteEvent[]` (the unified note list for both melodic and drum clips), optional `ClipEnvelope[]` for per-clip automation, and an optional `sample` field for loop/song audio clips. Clips also carry `loopEnabled` / `loopStartTick` / `loopEndTick` for sub-region looping, and a `gridResolution` hint for the drum editor.
- **`SessionScene`** — a `clipPerLane` map from lane id to clip slot index (or null for a stopped lane).

Notes carry a `velocity` field (0–127). The `velToColor` function in `src/core/velocity-color.ts` maps velocity to a blue-to-yellow ramp used by both the piano roll and the drum grid.

Saves are written as `schemaVersion: 3` (`SavedStateV3` in `src/save/`). Older saves are normalised by `session-migration.ts` at load time before anything else touches the data.

LaneResourceMap and the audio graph

The master audio path assembled in `src/app/audio-graph.ts` runs:

```
master GainNode → insert chain → MasterCompressor → AnalyserNode
                                                   → SidechainBus
```

Each lane's `LaneResources` consists of a `ChannelStrip` (level, EQ, send levels), a `SynthEngine`, and an `InsertChain` of per-lane FX. `LaneResourceMap.replaceEngine` hot-swaps only the engine while keeping the strip and inserts in place — the channel-level resources survive an engine swap.

`ensureLaneResource` in the lane allocator is the only place that constructs a `LaneResources`. Call it once per lane before accessing anything in the map. Test code that needs a lane wired up must call it explicitly as setup.

The scheduler

The `Sequencer` class (`src/core/sequencer.ts`) is a 25 ms look-ahead timer. On each tick it calls `sessionTick(now, lookaheadSec)`, and the session host fans that out to `tickLane` for each playing lane.

`tickLane` (`src/core/lane-scheduler.ts`) implements the Chris Wilson two-clocks pattern: for every `NoteEvent` whose absolute schedule time falls in the window `[now, now + lookaheadSec)`, it calls `ctx.onTrigger`. Schedule times are derived by converting clip-tick positions to seconds using the current BPM and projecting onto the absolute timeline from the loop-start anchor. Step duration for a 16th note is `60 / bpm / 4` seconds.

Two important consequences for contributors:

- `bpm` and `length` are mutable at runtime; the next scheduled step picks up the new values immediately.
- Engine params are read at trigger time, not when a note is held. Live knob tweaks apply to the **next** trigger.

When `clip.loopEnabled` is set, `effectiveClipLoop` (`src/core/clip-loop.ts`) constrains the iteration window to `[loopStartTick, loopEndTick)`. The brace UI in `src/core/clip-loop-brace.ts` is the editing surface for that region.

How-to recipes

Add a synth engine

1. Create `src/engines/<name>.ts`. Implement `SynthEngine` and export a `PluginFactory` with `kind: 'synth'`.
2. Call `registerEngine(instance)` and `registerEngineFactory(id, () => new YourEngine())` at module scope.
3. Declare params as `EngineParamSpec[]` and implement `getAudioParams()` on each voice so modulation and automation work automatically.
4. That is it. The build-time glob discovers the file; it appears in the lane engine selector without any further wiring.

See Engines for the full engine catalogue.

Add an FX insert or modulator

1. Create `src/plugins/fx/<name>.ts` or `src/plugins/modulators/<name>.ts`.
2. Export a `PluginFactory` with `kind: 'fx'` or `kind: 'modulator'` respectively, and call `registerPlugin` at module scope.
3. FX inserts mount per-lane and on master automatically; modulators appear in the modulation panel. See Modulation and Note FX for the user-facing side.

Add a preset

Open `public/presets/<engine>.json` and append an entry. The `gm` field is optional (an integer GM program number for MIDI-import matching). JSON is the source of truth; `preset-loader.ts` validates and `preset-apply.ts` applies it at runtime by calling `engine.applyPreset`. Each engine's JSON keys are its own vocabulary — do not use a generic `setBaseValue` loop.

Add a synth drum kit

Append an object to the `KITS` array in `src/core/drums.ts`. Kits are parameter bags over shared DSP primitives. To add a new drum *voice* (not just a new kit): extend the `DrumVoice` union, add it to `DRUM_LANES`, add an entry to every kit, implement a `play<Voice>()` method, and add a `trigger()` case.

Add a sampled drum kit

1. Create a subdirectory `public/drumkits/<id>/` containing WAV files for each voice (e.g. `kick.wav`, `snare.wav`, `closedHat.wav`).
2. Add a manifest file `public/drumkits/<id>.json` with `id`, `name`, and a `samples` array. Each entry needs `voice`, `note` (GM MIDI note number), and `file` (path relative to `public/drumkits/`).
3. Register the kit in `public/drumkits/index.json` by appending `{ "id": "<id>", "name": "<display name>" }`.

The existing kits (`tr808`, `acoustic`, `dirt`) follow this layout exactly and are the reference.

Source layout tour

```
src/
  core/           DSP primitives + pure logic (synth, drums, sequencer,
                  lane-scheduler, lane-resources, fx, meter, notes,
                  history, knob, pianoroll, …)
                  velocity-color.ts / velocity-gain.ts / velocity-lane-editing.ts
                    — note-velocity colour ramp, gain curve, lane editing helpers
                  clip-loop.ts / clip-loop-brace.ts
                    — clip sub-region resolver + drag-brace UI primitive
  engines/        SynthEngine abstraction, registry, one file per engine
                  (tb303, subtractive, fm, wavetable, karplus, sampler,
                  drums-engine) + engine-selector UI
  session/        SessionState model + all session UI
                  (session-host, session-ui, session-inspector,
                  clip-editors/, session-migration)
  modulation/     LFO/ADSR voices, ModulationHost, ModulatorScope,
                  connection binder
  plugins/        Plugin SPI + registry
                  fx/       — multifilter, distortion, reverb, delay, InsertChain
                  modulators/ — lfo, adsr
  presets/        Preset loader + apply logic
                  (JSON assets live in public/presets/)
  midi/           SMF parser, MIDI-to-session transform, GM lookup, import UI
  samples/        Sample types, IndexedDB store, buffer cache, keymap,
                  import metadata
  stems/          Stem-separation client + config + lane-plan builder
                  (talks to the local Python service in tools/stem-service/)
  performance/    Arrangement / record model:
                  arrangement-from-session, arrangement-ops,
                  arrangement-runtime (records clip-launches + knob automation;
                  surfaced via the REC group's take mode — see performance-feature)
  polysynth/      PolySynth poly voice host (used by Subtractive):
                  voice stealing, mono/legato/retrig, per-voice mod bus
  app/            Boot factories: audio-graph, lane-allocator, trigger-dispatch,
                  knob-mounting, mute-solo, bpm-broadcast, engine-swap,
                  plugin-bootstrap, lane-host-wiring
  save/           SaveManager (schemaVersion: 3), history-wiring (withUndo)
  notefx/         Note-FX plugin category (arpeggiator, chord spread) — per-lane
  automation/     Clip envelope recording + read-back helpers
  arp/            Arpeggiator voice logic
  copy/           Clipboard helpers for clip/note copy-paste
  demo/           Baked MIDI demos + demo picker
  styles/         SCSS

public/
  presets/        Engine preset JSONs (20+ per engine, GM-tagged)
  drumkits/       Sampled drum kits: index.json + <id>.json manifests + WAVs

tools/
  stem-service/   Local Python service (FastAPI + audio-separator / Demucs)
```

```
                exposing an HTTP job queue for stem separation.
                Run: uvicorn app:app --port 8765
                Tests: python -m pytest test_app.py (not part of npm test)
```

Testing

Loom has four test layers, one per risk class.

Pure logic (`src/**/*.test.ts` , excluding `.dsp` and `.wiring` suffixes) — schemas, scales, migrations, session/arrangement logic, modulation math. These run fast and have no audio dependencies.

Scheduling with a fake clock — `src/core/lane-scheduler.test.ts` and `src/session/session-runtime.test.ts` drive the look-ahead scheduler through a mock `AudioContext` clock. The fake clock advances in controlled steps so timing edge-cases are deterministic.

Real DSP (`*.dsp.test.ts`) — every engine and drum kit is rendered through `OfflineAudioContext` via the `node-web-audio-api` package, globalised in `test/setup.ts` . Use the shared battery `runStandardEngineBattery` from `test/dsp-battery.ts` . Each render writes a WAV to `test/output/` (gitignored). Compare against the committed reference in `test/golden/` with `npm run test:wav-diff` ; promote with `npm run test:wav-bless` .

Modulation wiring (`*.wiring.test.ts`) — LFO/ADSR voices connected through a depth-gain bridge into a target `AudioParam` , verified end-to-end.

Assertion rule: always write relative assertions (`a > b` , `a > b * 2`). Never hard-code absolute magnitudes — they are a brittleness smell. If you must write one, justify it in a comment.

Colour-free output: every `npm test` script runs under `cross-env NO_COLOR=1` . When invoking Vitest directly, prefix with `NO_COLOR=1` . Do not add `--reporter=...` — the scripts already configure the right reporter.

Key commands:

Command	What it runs
`npm run dev`	Vite dev server with hot reload at http://localhost:5173
`npm run build`	`tsc` typecheck + Vite bundle to `dist/`
`npm test`	Full suite: unit + e2e (always build first)
`npm run test:unit`	Vitest only, no browser
`npm run test:fast`	Unit tests excluding DSP renders (inner-loop TDD)
`npm run test:dsp`	DSP renders only (slow; needs `node-web-audio-api`)
`npm run test:e2e`	Playwright against `vite preview` on port 4173

e2e gotcha: `test:e2e` and `npm test` serve `dist/` with no build step. Playwright boots `vite preview` over the last production bundle. If you changed `src/` without rebuilding, the newest features are absent from the bundle and tests fail with "element not found" — which looks like a regression. Always run `npm run build` before `npm run test:e2e`.

Vitest runs test files serially (`fileParallelism: false`) because `node-web-audio-api`'s `OfflineAudioContext` is not safe under parallel forks. The teardown occasionally exits non-zero with `ERR_IPC_CHANNEL_CLOSED` after all tests pass — that is a tinypool shutdown race, not a test failure; re-run to confirm green.

For the definitive, always-up-to-date architecture reference, read `CLAUDE.md` at the repo root. Implemented design docs are intentionally removed from the tree once shipped (they drift); recover rationale from git history when you need it. Unfinished work lives in `docs/superpowers/REMAINING-WORK.md`.